

X-Ray Analyzer — How the AI Works

Overview

The AI system is a **multi-stage pipeline** that combines deep learning computer vision (PyTorch DenseNet121), explainability (Grad-CAM), and LLM-powered natural language generation (Groq API). Together they convert a raw X-ray image into:

- A **medical prediction** with confidence score
- A **heatmap overlay** showing what the model looked at
- **Patient-friendly** and **clinical-grade** explanations

Architecture at a Glance



Stage 1 — Image Preprocessing

File: [preprocessing.py](file:///c:/Users/Jithi/Desktop/X-Ray%20Analyzer/backend/routes/ai/preprocessing.py)

The raw X-ray is transformed into a tensor the model can consume:

```
transforms.Compose([
    transforms.Resize((224, 224)), # DenseNet input size
    transforms.Grayscale(num_output_channels=3), # X-rays are grayscale → replicate to 3ch
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], # ImageNet stats
                        std=[0.229, 0.224, 0.225]),
])
```

- X-rays are inherently grayscale, but DenseNet expects 3-channel RGB, so they are replicated across 3 channels.
- ImageNet normalization is used because the backbone was pretrained on ImageNet.
- Output: tensor [1, 3, 224, 224]

Stage 2 — Model Inference (DenseNet121)

File: [inference.py](file:///c:/Users/Jithi/Desktop/X-Ray%20Analyzer/backend/routes/ai/inference.py)

Model Architecture

```
DenseNet121 (ImageNet backbone)
└─ Replaced final classifier head:
    nn.Linear(1024 → num_classes)
    where num_classes = number of conditions per body part
```

Body Part Registry

File: [body_parts.py](file:///c:/Users/Jithi/Desktop/X-Ray%20Analyzer/backend/routes/ai/body_parts.py)

12 distinct body parts are supported, each with its own fine-tuned .pth model file:

Body Part	Detectable Conditions
Chest	Normal, Pneumonia
Knee	Normal, Osteoarthritis, Fracture, Effusion
Hand & Wrist	Normal, Fracture, Arthritis, Bone_Cyst
Spine	Normal, Scoliosis, Compression_Fracture, Disc_Narrowing, Spondylolisthesis
Hip & Pelvis	Normal, Fracture, Hip_Dysplasia, Osteoarthritis, Avascular_Necrosis
Shoulder	Normal, Fracture, Dislocation, AC_Joint_Separation
Elbow	Normal, Fracture, Dislocation, Arthritis
Foot & Ankle	Normal, Fracture, Arthritis, Flat_Foot
Forearm	Normal, Fracture, Bowing
Lower Leg	Normal, Fracture, Stress_Fracture, Bone_Lesion
Skull	Normal, Fracture, Sinusitis
Abdomen	Normal, Bowel_Obstruction, Free_Air, Calcification

Model Loading Strategy

Models are **lazily loaded and cached** in memory:

1. Check local models/<body_part>.pth file
2. If missing → auto-download from **Hugging Face Hub** (jithinjz/xray-models)
3. If HF unavailable → fall back to ImageNet pretrained weights (lower accuracy)

Inference Output

```
{
  "body_part": "knee",
  "prediction": "Osteoarthritis",
  "confidence": 0.8732,
  "probabilities": {
    "Normal": 0.08,
    "Osteoarthritis": 0.87,
    "Fracture": 0.03,
    "Effusion": 0.02
  }
}
```

Stage 3 — Grad-CAM Heatmap

File: [gradcam.py](file:///c:/Users/Jithi/Desktop/X-Ray%20Analyzer/backend/routes/ai/gradcam.py)

Grad-CAM (Gradient-weighted Class Activation Mapping) highlights **which pixels of the X-ray** caused the prediction. This is what makes the AI explainable.

How it works (step by step):

1. **Hook denseblock4** — the last convolutional block in DenseNet121 before global average pooling
2. **Forward pass** — get model output, capture intermediate feature maps
3. **Backward pass** — compute gradients of the predicted class score w.r.t. the feature maps
4. **Global Average Pool gradients** → importance weight per feature channel

```
weights = grad.mean(dim=(2, 3), keepdim=True) # [1, C, 1, 1]
```

5. **Weighted sum of feature maps** → raw heatmap

```
cam = (weights * act).sum(dim=1) # [1, 1, H, W]
cam = torch.relu(cam)           # keep only positive influence
```

6. **Normalize** to [0–255], **resize** to original X-ray dimensions
7. **Apply JET colormap** (blue=low activation, red=high activation)
8. **Blend** heatmap (40%) over original X-ray (60%) → overlay PNG saved to disk

Stage 4 — LLM Explanations via Groq

File: [llm.py](file:///c:/Users/Jithi/Desktop/X-Ray%20Analyzer/backend/services/llm.py)

Once inference is complete, the results are sent to **Groq's API** using llama-3.3-70b-versatile to generate two separate outputs:

4a — Patient Summary (JSON)

Prompt role: *"Expert AI radiologist communicating with a patient"*

The LLM is asked to produce a structured JSON object:

```
{
  "headline": "Possible lung infection detected",
  "emoji": " 🦠 ",
  "what_found": "... (6th-grade reading level) ...",
  "what_it_means": "...",
  "what_to_do": "...",
  "urgency": "urgent | soon | watch | good",
  "confidence_text": "The AI is very confident in this result (87%).",
  "body_part_label": "Chest / Thorax",
  "prediction": "Pneumonia"
}
```

- Temperature: **0.3** (slightly creative but mostly factual)
- Response format: enforced JSON object

4b — Clinical Notes (Markdown)

Prompt role: *"Expert thoracic and skeletal radiologist writing clinical notes for the referring physician"*

The LLM generates professional markdown-formatted clinical notes:

```
### Findings Summary
- AI prediction: Pneumonia (87% confidence)
- ...

### Differential Diagnosis & Significance
- ...

### Recommendations
- ...
```

- Temperature: **0.2** (very conservative/precise)
- Falls back to local template if Groq API key is missing or request fails

Stage 5 — Orchestration

File: [service.py](file:///c:/Users/Jithi/Desktop/X-Ray%20Analyzer/backend/routes/ai/service.py)

The analyze_scan() function ties all stages together:

```
1. Validate body part exists in registry
2. Fetch scan from MongoDB, verify image was uploaded
3. Mark scan status → "processing" in DB
4. preprocess_image() → tensor
5. run_inference() → prediction, confidence, probabilities
6. generate_gradcam() → heatmap PNG saved to disk
7. get_patient_summary() → local template fallback
8. generate_llm_explanations() → Groq LLM enhanced output
9. save_ai_result() → persist to MongoDB, status → "analyzed"
```

Idempotent: If a scan is already analyzed for the same body part, the cached result is returned immediately.

API Endpoints

File: [router.py](file:///c:/Users/Jithi/Desktop/X-Ray%20Analyzer/backend/routes/ai/router.py)

Method	Endpoint	Description
GET	/api/v1/ai/body-parts	List all 12 supported body parts + their conditions
POST	/api/v1/ai/analyze	Trigger full AI analysis for a scan
GET	/api/v1/ai/gradcam/{scan_id}	Serve the Grad-CAM heatmap PNG

All endpoints require a **doctor** role JWT token.

Key Configuration (.env)

Variable	Purpose
GROQ_API_KEY	Enables LLM explanations via Groq (without it, local templates are used)
GROQ_MODEL	Default: llama-3.3-70b-versatile
HF_MODEL_REPO	Default: jithinjz/xray-models — where .pth weights are hosted
HF_TOKEN	Required for private HF repos

Summary of AI Technologies Used

Component	Technology
Image Classification	DenseNet121 (PyTorch / torchvision)
Explainability	Grad-CAM on denseblock4
Model weights storage	Hugging Face Hub
Patient-language generation	Groq API (llama-3.3-70b-versatile)
Clinical notes generation	Groq API (llama-3.3-70b-versatile)
Inference device	CUDA if available, otherwise CPU